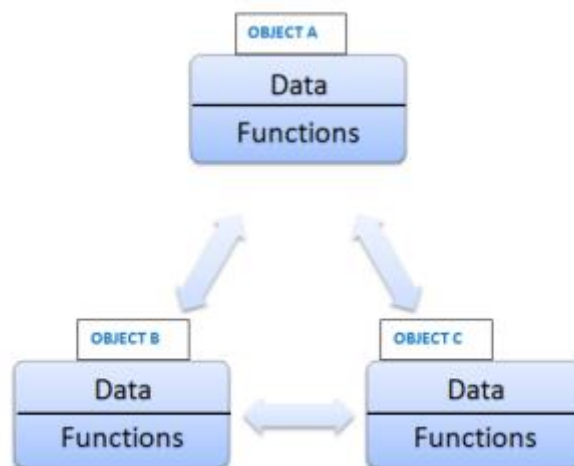


UNIT I: Introduction to OOP, procedural programming language and object oriented language, principles of OOP, applications of OOP, history of java, java features, JVM, program structure. Variables, primitive data types, identifiers, literals, operators, expressions, precedence rules and associativity, primitive type conversion and casting, flow of control.

Introduction to OOP:

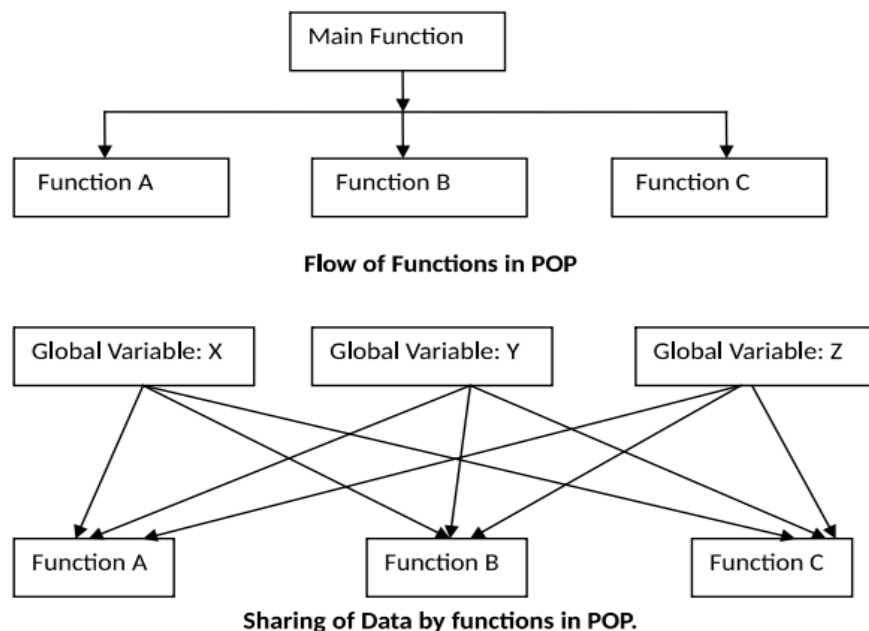
- The major motivating factor in the invention of object-oriented approach is to remove drawbacks of procedural oriented approach.
- OOP treats data as critical element in the program development and does not allow it freely to flow around the system or application or program.
- OOP combines the data and methods(functions) into a single unit this process is called encapsulation.
- The organisation of data and Methods(functions) in object-oriented programming is shown in the following diagram.



- OOP allows divide a problem into a no of entities called as objects and then build data and Methods(functions) around these objects.
- The data of an object can be accessed only by the method(function) associated with that object.
- In OOP, method(function) of one object can access the method(function) of other objects.

Disadvantage of Conventional Programming:

- Traditional programming languages such as COBOL, FORTRAN, C etc. are commonly known as procedure-oriented languages and also called as Conventional Languages.
- The program written in these languages consists of a sequence of instructions that tells the compiler or interpreter to perform a given task.
- When a program code is large, it becomes inconvenient to manage and debug. To overcome this problem, it is divided into smaller functions to carry out some specific task.
- Each function can call another function during the execution.
- As the functions are executing, they may access same data, and modify the data which in turn affects the entire task.
- Most of the functions are allowed to access the global data.

**Disadvantages:**

1. Large programs are divided into smaller programs known as functions. These functions can call one another. Hence security is not provided.
2. No importance is given to security of data and importance is given to doing things.
3. Data move openly around the system from function to function.
4. Most functions access global data, what happens to the data is not known.

Difference between POP and OOP:

Following are the differences Between POP and OOP

S.No.	POP	OOP
1	POP means Procedure Oriented Programming language.	OOP means Object Oriented Programming language.
2	It is platform dependent language.	It is platform independent language.
3	POP uses Top-down approach in Program Design.	OOP uses Bottom-up approach in Program Design.
4	In POP large program is divided into functions.	In OOP entire program is divided into objects.
5	Global data is shared among the functions in the program.	Data is shared among the objects through the methods.
6	Instead of focusing on data, POP focuses on method or procedure.	OOP mainly focuses on data security instead of focusing on method or procedure.
7	POP does not support access modifiers.	OOP supports access modifiers.
8	POP does not support object-oriented concepts.	OOP supports object-oriented concepts like Class, Object, Encapsulation and Data hiding, Inheritance, Polymorphism, Abstraction and Message Passing.
9	POP does not support Interfaces, Multithreading and Packages concepts.	OOP supports Interfaces, Multithreading and Packages concepts.
10	POP does not support Exception handling and Event handling.	OOP supports Exception handling and Event handling.
11	Uses compiler only	Uses compiler and interpreter
12	Example: C, VB, FORTRAN, PASCAL	Example: C++, JAVA, VB.NET, C#.NET

Difference between C and Java:

Following are the differences Between C and Java:

S.No.	C	Java
1	C is Procedure Oriented Programming language	Java is an Object-Oriented Programming language.
2	C was developed by Dennis Ritchie in 1972.	Java was developed by James Gosling in 1995
3	It is platform dependent	It is platform independent

4	Top-down approach is used in Program Design.	Bottom-up approach adopted in Program Design.
5	In C • scanf() Function used for Input. • printf() Function used for output.	In Java • System.out.println() used for output. • System.in used from input.
6	In C format specifiers are required to read and print the data.	In Java format specifiers are optional or not required to print or read the data.
7	C uses “#include <stdio.h>” header file for input and output operations.	Java uses “import java.io.*;” for the same purpose.
8	In C all variables are declared at the starting of the scope.	In Java all variables are declared anywhere in the scope. i.e.at the time of first use.
9	C is a function-driven language.	Java is an object-driven language
10	Instead of focusing on data, C focuses on method or procedure.	Java focuses on data security instead of focusing on method or procedure.
11	In C, malloc() and calloc() Functions are used for Memory Allocation and free() function for memory Deallocating.	In Java, new is used for Memory Allocation and Deallocation is done by the JVM.
12	C contains 32 keywords.	Java contains 50 keywords.
13	C does not support object-oriented concepts.	Java supports object-oriented concepts like Class, Object, Encapsulation and Data hiding, Inheritance, Polymorphism, Abstraction and Message Passing.
14	C does not support Interfaces, Multithreading and Packages concepts.	Java supports Interfaces, Multithreading and Packages concepts.
15	C does not support access modifiers.	Java supports access modifiers.
16	C does not support constructors.	Java supports constructors.
17	C does not support Exception handling and Event handling.	Java supports Exception handling and Event handling.
18	The extension of C program is “.c”.	The extension of Java program is “.java”
19	Uses compiler only.	Uses compiler and interpreter.

Difference between C++ and Java:

Following are the differences Between C++ and Java:

S. No	C++	Java
1	C++ was developed by Bjarne Stroustrup in 1979.	Java was developed by James Gosling in 1995.
2	C++ is not a purely object oriented language, since it is possible to write C++ programs without using a class or object.	Java is purely an object oriented language, since it is not possible to write java programs without using atleast one class.
3	It is platform dependent.	It is platform independent.
4	Uses compiler only	Uses compiler and interpreter
5	C++ uses cin for input and cout for an output operation.	Java uses the (System class): System.in for input and System.out for output.
6	It doesn't support documentation comments for source code.	It supports documentation comments (e.g., <code>/**.. */</code>) for source code.
7	C++ supports goto keyword.	Java doesn't support goto Keyword
8	C++ supports Structures and Unions.	Java doesn't support Structures and Unions.
9	It has 63 keywords	It has 50 keywords
10	Supports Multiple inheritance in C++.	Doesn't support multiple inheritance in java, but we can achieve this by interfaces.
11	There are constructors and destructors in C++.	Only constructors are available in Java.
12	It supports call by value and call by reference	It supports call by value only
13	It supports both method and operator overloading.	It supports only method overloading and doesn't allow operator overloading.
14	It has Virtual Keyword	It doesn't have Virtual Keyword.
15	It supports manual object management using new and delete.	Automatic object management with garbage collection.
16	Execution can be done by OS.	Execution can be done by JVM.
17	C++ doesn't support >>> operator.	Java supports >>> operator.
18	The extension of C++ program is ".cpp".	The extension of Java program is

		“.java”
--	--	---------

Basic Concepts (Key Concepts) / principles / features of Object-Oriented Programming:

Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.

It simplifies the software development and maintenance by providing some concepts like:

- Class
- Object
- Encapsulation & Data hiding
- Inheritance
- Polymorphism
- Abstraction
- Message Passing

Class:

Class is an abstract data type (user defined data type) / non-primitive data type that contains attributes and methods that operate on data. It starts with the keyword class.

(or)

Class is a blueprint or template for creating an object.

```
e.g.: public class Student
{
    String name;
    int id;
    public void readData()
    {
    }
    public void displayData ()
    {
    }
};
```

Object:

An object is an instance of a class. It is a variable that represents attributes as well as methods required for operating on the data. They interact with private data and methods through public methods.

(or)

An object is an entity that exists physically in the real world which requires some memory will be called as an object.

An object consists of:

State: It is represented by attributes / properties of an object.

Behaviour: It is represented by actions / methods of an object.

e.g.: `Student s1 = new Student ();`

`Student s2 = new Student ();`

In the above example **Student** is the class name and s1 and s2 are objects of that class.

e.g.: Class	Object
Fruit	Apple, Banana, Grape
Animal	Lion, Tiger, Elephant
Mobile	Apple, Samsung, Redmi, Realme
Laptop	Dell, HP, Lenovo, Mi

Encapsulation & Data hiding:

Encapsulation:

Encapsulation is the process of combining data and methods into a single unit is called as encapsulation.

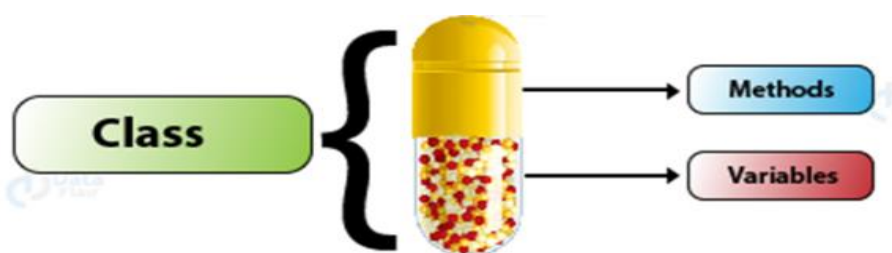
Data Hiding:

Data hiding means hiding the internal data within the class to prevent its direct access from outside the class. It is achieved by using an access modifier - a private modifier.

i.e., Private data and methods are available to the public methods only. They cannot be accessed by the other part of the program. This process of hiding private data and methods from the other part of the program is called as data hiding

Note: The recommended modifier for data members is private. The main advantage of data hiding is security

Example 1: capsule, it is wrapped with different medicines.

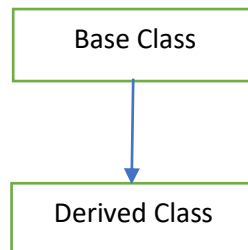


Example 2:

```
class Account {
private double account_balance;
.....
.....
}
```

Inheritance:

- It is a process of using the properties of one class inside other class is known as inheritance.
- The main aim of Inheritance is **Reusability** of the data and methods i.e. data and methods declared in one class can be accessed by other class.
- In Inheritance, two classes are involved (**super class/base class/parent class and sub class/derived class/child class**).
- The class which gives the properties is called as super class / base class / parent class.
- The class which takes the properties and methods is called as sub-class/derived class /child class.



Reusability: Using the already existing code is called as reusability. This is mostly used in inheritance. The already existing code is inherited to the new class. It saves a lot of time and effort. It also reduces the size of the program.

Polymorphism:

The word polymorphism is derived from two latin words poly(many) and morph (forms).

Polymorphism means the ability to take many forms. Polymorphism allows to take different implementations for same name.

Poly -> many

Morph -> forms

ism -> behaviours

There are two types of polymorphism, Compile time polymorphism and Run time polymorphism.

In Compile time polymorphism binding is done at compile time and in runtime polymorphism binding is done at runtime.

e.g.: Method Overloading and Method Overriding

Method Overloading:

- Method overloading is used for compile time polymorphism.
- Same method name having different implementations with different number and type of arguments.

Method Overriding:

- Method overriding is used for runtime polymorphism

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.

e.g.: mouseClicked()

Close button -> Window gets closed
Max button -> window gets maximised
Min button -> window gets minimized

Abstraction:

It is a process of Hiding **internal details and showing functionality** is known as abstraction.

For example: Phone call, we don't know the internal processing.

Message Passing:

An object-oriented program contains a set of objects that communicate with one another. The process of object-oriented programming contains the basic steps:

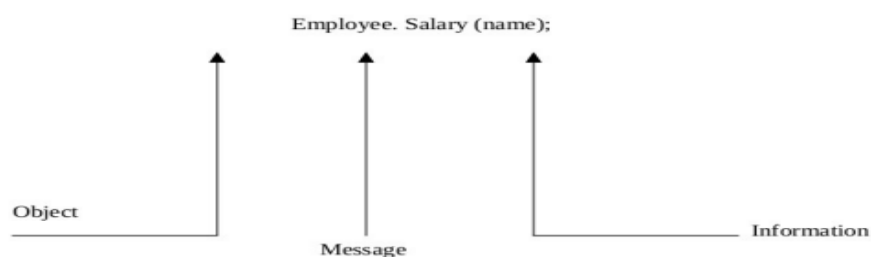
1. Creating classes
2. Creating objects
3. Establishing communication among objects

This communication is done with the help of methods (i.e., passing objects to methods)

Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another.

Message passing involves specifying the name of object, the name of the method (message) and the information to be sent.

Example:



Advantages of OOP:

Object oriented technology provides many advantages to the programmer and the user. This technology solves many problems related to software development, provides improved quality and low-cost software.

1. Object oriented programs can be comfortably upgraded.
2. Using inheritance, we can eliminate redundant program code and reuse already existing code.
3. Data hiding facilitates the programmer to design and develop safe programs.
4. Through polymorphism, it provides the extensibility of the code.
5. With OOP, programs are easy to understand.
6. With OOP, programs are easier to test, manage and maintain.
7. Objects can communicate with each other by message passing mechanism.

Applications of OOP:

The promising areas for application of OOP includes:

1. Real-time systems
2. Simulation and modelling
3. Object-oriented databases
4. Hypertext, hypermedia and expert text
5. AI and expert systems
6. Neural networks and parallel programming
7. Decision support and Automation system
8. CIM/CAM/CAD systems

History of java:

1. Java is an Object-Oriented Programming language, High level, Case Sensitive and General purpose programming language.
2. JAVA was developed by James Gosling and his team at Sun Microsystems Inc in the year 1991, later acquired by Oracle Corporation.
3. James Gosling is known as the father of Java.
4. Before Java, it was called Oak and was developed as a part of the Green project. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java In 1995.
5. Java is an island in Indonesia where the first coffee was produced (called Java coffee). Java name was chosen by James Gosling while having a cup of coffee nearby his office.
6. Java is just a name, not an acronym.

Features (Buzzwords) of Java:

Java has become a popular language because of the following features:

- Simple
- Object-Oriented
- Portable
- Platform independent
- Secured
- Robust
- Architecture neutral
- Compiled and Interpreted
- High Performance
- Multithreaded
- Distributed
- Dynamic

1. **Simple:** Java is one of the simple languages as it does not have complex features like pointers, operator overloading, multiple inheritances, and explicit memory allocation.
2. **Object-Oriented:** Java Supports OOP features like Class, Object, Encapsulation and Data hiding, Abstraction, Inheritance, Polymorphism, Message Communication/ Message Passing.
3. **Portable:** Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.
4. **Platform independent:** Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere (WORA).
5. **Secured:** Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

No explicit pointer

Java Programs run inside a virtual machine sandbox
6. **Robust:** Java language is robust that means reliable.

The main features of java that make it robust are garbage collection, Exception Handling, and memory allocation.
7. **Architecture neutral:** Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 16-bit architecture and 4 bytes of memory for 32 and 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

- 8. Compiled and Interpreted:** Unlike other languages Java uses a system with both a compiler and an interpreter for program execution. First, the compiler converts the program code to bytecode which in turn is converted to machine code on any machine, using the interpreter. The machine code thus generated can be executed irrespective of the system on which it is being executed.
- 9. High Performance:** Java architecture is defined in such a way that it reduces overhead during the runtime and at some time java uses Just In Time (JIT) compiler where the compiler compiles code on-demand basics where it only compiles those methods that are called making applications to execute faster.
- 10. Multithreaded:** A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.
- 11. Distributed:** We can create distributed applications using the java programming language. Remote Method Invocation and Enterprise Java Beans are used for creating distributed applications in java. The java programs can be easily distributed on one or more systems that are connected to each other through an internet connection.
- 12. Dynamic:** Java being completely object-oriented gives us the flexibility to add classes, new methods to existing classes and even creating new classes through sub-classes. Java even supports functions written in other languages such as C, C++ which are referred to as native methods.

Applications of Java:

1. Desktop Applications such as acrobat reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics

8. Games, etc.

Java Versions:

Many java versions have been released till now. The current stable release of Java is Java SE 10.

1. JDK Alpha and Beta (1995)

2. JDK 1.0 (23rd Jan 1996)
3. JDK 1.1 (19th Feb 1997)
4. J2SE 1.2 (8th Dec 1998)
5. J2SE 1.3 (8th May 2000)
6. J2SE 1.4 (6th Feb 2002)
7. J2SE 5.0 (30th Sep 2004)
8. Java SE 6 (11th Dec 2006)
9. Java SE 7 (28th July 2011)
10. Java SE 8 (18th Mar 2014)
11. Java SE 9 (21st Sep 2017)
12. Java SE 10 (20th Mar 2018)
13. Java SE 11 (September 2018)
14. Java SE 12 (March 2019)
15. Java SE 13 (September 2019)
16. Java SE 14 (Mar 2020)
17. Java SE 15 (September 2020)
18. Java SE 16 (Mar 2021)

19. Java SE 17 (September 2021)

20. Java SE 18 (to be released by March 2022)

Note: Since Java SE 8 release, the Oracle corporation follows a pattern in which every **even version** is release in **March month** and an **odd version** released in **September month**.

Java Platforms / Editions:

There are 4 platforms or editions of Java:

1) Java SE (Java Standard Edition)

It is a Java programming platform. It includes Java programming APIs such as java.lang, java.io, java.net, java.util, java.sql, java.math etc. It includes core topics like OOPs, String, Regex, Exception, Inner classes, Multithreading, I/O Stream, Networking, AWT, Swing, Reflection, Collection, etc.

2) Java EE (Java Enterprise Edition)

It is an enterprise platform that is mainly used to develop web and enterprise applications. It is built on top of the Java SE platform. It includes topics like Servlet, JSP, Web Services, EJB etc.

3) Java ME (Java Micro Edition)

It is a micro platform that is dedicated to mobile applications.

4) JavaFX

It is used to develop rich internet applications. It uses a lightweight user interface API.

JVM (Java Virtual Machine):

JVM (Java Virtual Machine) is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed.

JVMs are available for many hardware and software platforms (i.e., JVM is platform dependent).

The JVM performs following operation:

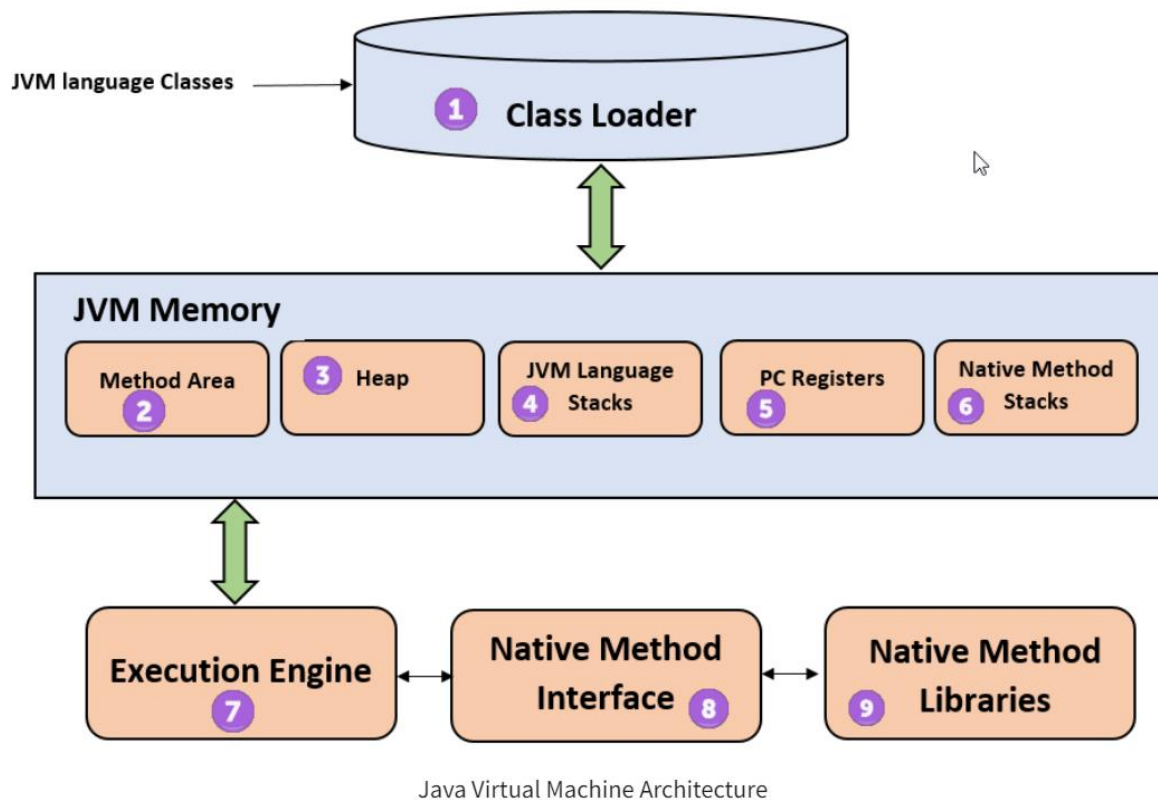
- Loads code
- Verifies code
- Executes code
- Provides runtime environment

Internal Architecture of JVM

the JVM is divided into three main subsystems:

1. Class Loader Subsystem
2. Java Runtime Subsystem
3. Execution Engine

1)



Class Loader

The class loader is a subsystem used for loading class files. It performs three major functions - Loading, Linking, and Initialization.

2) Method Area

In the method area, all class level information like class name, immediate parent class name, methods and variables information etc. are stored, including static variables. There is only one method area per JVM, and it is a shared resource.

3) Heap

Information of all objects is stored in the heap area. There is also one Heap Area per JVM. It is also a shared resource.

4) JVM language Stacks

Java language Stacks store local variables, and it's partial results. Each thread has its own JVM stack, created simultaneously as the thread is created. A new frame is created whenever a method is invoked, and it is deleted when method invocation process is complete.

5) PC Registers

PC register store the address of the Java virtual machine instruction which is currently executing.

6) Native Method Stacks

It contains all the native methods used in the application.

7) Execution Engine

It contains:

Interpreter: It interprets the bytecode line by line and then executes. The disadvantage here is that when one method is called multiple times, every time interpretation is required.

Just-In-Time (JIT) compiler: It is used to increase the efficiency of an interpreter. It compiles the entire bytecode and changes it to native code so whenever the interpreter sees repeated method calls, JIT provides direct native code for that part so re-interpretation is not required, thus efficiency is improved.

8) Native Method interface

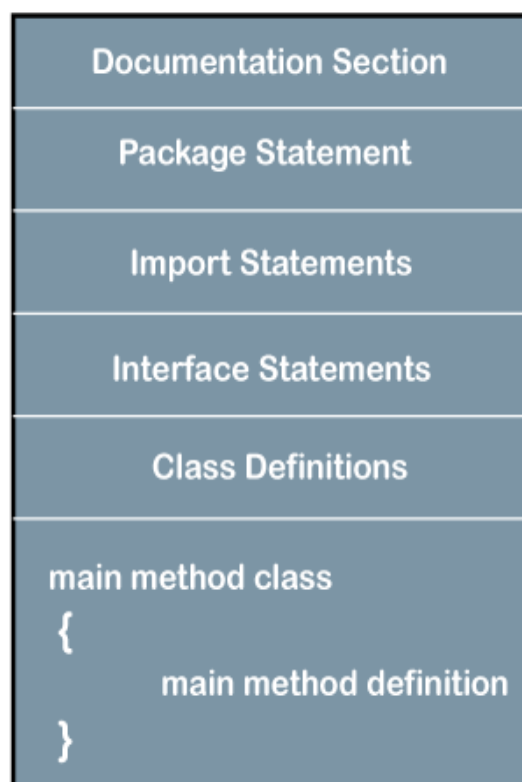
It is an interface that interacts with the Native Method Libraries and provides the native libraries (C, C++) required for the execution. It enables JVM to call C/C++ libraries and to be called by C/C++ libraries which may be specific to hardware.

9) Native Method Libraries

Native Libraries is a collection of the Native Libraries (C, C++) which are needed by the Execution Engine.

Java Program Structure:

A java program may contain one or more sections as shown in the figure.



Structure of Java Program

Documentation Section (Optional)

1. This section comes first and it is also called as comments sections.
2. Comments are non-executable statements in any programming language.
3. Comments are used to write the program statement / program name.
e.g. `/* Write a C++ program illustrating Checking whether the number is even or odd using Ternary operator. */`
4. Comments are used to write the program log.
e.g. `/**Program Description : To check the no is even or odd`

Created Date	: 28/10/2021
Created Time	: 10.00 am
Created by	: ABC
Owner	: XXX

`*/`
5. Comments are used to write the remarks for each line in the program.
6. Comments are 3 types in java
 1. **Single line Comments:**
Single-line comments start with two forward slashes (`//`).
Example: `// This is a comment`
 2. **Multi line Comments:**
Multi-line comments start with `/*` and ends with `*/`.
Example: `/* Write a C++ program illustrating Checking whether the number is even or odd using Ternary operator. */`
 3. **Documentation Comment:** It starts with the delimiter (`/**`) and ends with `*/`.
7. Whatever written in the documentation section is the comment and is not compiled by the compiler.
8. Documentation Section is optional since the program can execute without them.

Package statement (Optional)

The first statement in a java file is a package statement. It declares a package name and inform the compiler that the classes define belong to the same package. This statement is optional.

```
Package package_name:
Package pkg1
{
.....
}
Package pkg2
{
.....
}
```

Import statement (optional)

These are similar to `# include` statement in C or C++, this instructs the interpreter to load the method or class contained in the package.

Interface statement (optional):

It is a class including a group of method declaration. It is used when multiple inheritances are implementing.

Class definition

Classes are defining to make the objects of real-world problem. The number of classes used depends on complexity of the program

Main method class:

It is the starting point and essential part of java programming while executing. It creates object of various classes and established a communication between them.

Simple Java Program

```
public class Demo {  
    public static void main (String args[]) {  
        System.out.println("Welcome to Java Class");  
    }  
}
```

Let us discuss the program line by line and understand the unique features that constitute a Java program.

Class Declaration

The first line class Demo, class is a keyword used to declare the class. As we know Java is a truly object-oriented language so everything must be placed inside a class. Demo is the identifier in java that defines the name of the class.

Opening brace

Every class definition in java begins with an opening brace “{” and ends with a matching closing brace “}”.

The Main Line:

```
public static void main (String args[])
```

Every Java program must include the main() method. Java applets does not use main() method. This line contains a number of keywords public, static and void.

Public:

The keyword is an access modifier which represents visibility; it means it is accessible to all other classes. This is similar to C++ public modifier.

Static:

Statics a keyword, if we declare any method as static, it is treated as static method. The static method invoked without creating instance of a class. The main method is executed by the JVM, so it doesn't require creating an object to call the main method. So, it saves memory.

void:

void means nothing. It is the return type. main method use void as it does not return any value.

String args[]:

main method accept some data from user as command line argument in form of String and this can be achieved by String args[] array.

Identifiers in Java:

Identifiers in Java are symbolic names used for identification. They can be a class name, variable name, method name, package name, constant name, and more.

Rules for Identifiers in Java

If the identifiers are not properly declared, we may get a compile-time error. Following are some rules and conventions for declaring identifiers:

- A valid identifier must have characters [A-Z] or [a-z] or numbers [0-9], and underscore (_) or a dollar sign (\$).

For example, @sample is not a valid identifier because it contains a special character which is @.

- There should not be any space in an identifier.

For example, Sample Program is an invalid identifier.

- An identifier should not contain a number at the starting.

For example, 123Sample is an invalid identifier.

- An identifier should be of length 4-15 letters only. However, there is no limit on its length. But it is good to follow the standard conventions.

- We can't use the Java reserved keywords as an identifier such as **int, float, double, char**, etc.

For example, int double is an invalid identifier in Java.

- An identifier should not be any query language keywords such as **SELECT, FROM, COUNT, DELETE**, etc

Example:

```
public class SampleProgram{  
    public static void main (String args[]) {  
        System.out.println("Welcome to Java Class");  
    }  
}
```

From the above example, we have the following Java identifiers:

1. SampleProgram (Class name)
2. main (User defined method)
3. String (Predefined class name)
4. args (String variables)
5. System (Predefined class name)
6. out (Instance name)
7. println (Predefined method name)

Valid identifiers:

- TestVariable
- testvariable
- a
- i
- Test_Variable
- _testvariable
- \$testvariable
- sum_of_array
- TESTVARIABLE
- TEST123
- Javatest123

Invalid identifiers:

- Test Variable (We cannot include a space in an identifier)
- 123javatest (The identifier should not begin with numbers)
- java+test (The plus (+) symbol cannot be used)
- a-javatest (Hyphen symbol is not allowed)
- java_&_Ttest (ampersand symbol is not allowed)
- Java'test (we cannot use an apostrophe symbol in an identifier)

Naming Conventions

Naming conventions specify the rules to be followed by a Java programmer while writing the names of packages, classes, methods etc.

1. Package names are written in small letters.

e.g.: java.io, java.lang, java.awt etc

2. Each word of class name and interface name starts with a capital

e.g.: Sample, AddTwoNumbers

3. Method names start with small letters then each word start with a capital

e.g.: sum (), sumTwoNumbers (), minValue ()

4. Variable names also follow the same above method rule

e.g.: sum, count, totalCount

5. Constants should be written using all capital letters

e.g.: PI, COUNT

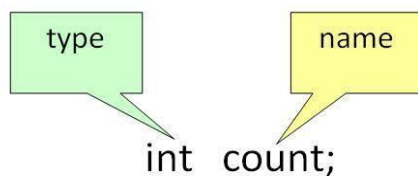
6. Keywords are reserved words and are written in small letters.

e.g.: int, short, float, public, void

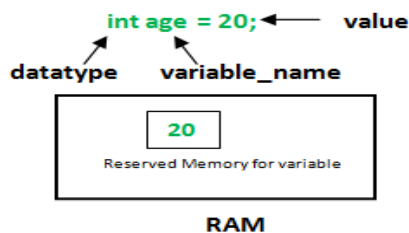
Variables:

- A variable is a name given to a memory location that is used to store the value.
- The value stored in a variable can be changed during program execution.
- In Java, all the variables must be declared before use.

Variable Declaration:



Variable initialization:



Types of Variables in Java

There are three types of variables in Java:

1. Local Variables
2. Instance Variables
3. Static Variables

1. Local Variables

- A variable defined within a block or method or constructor is called a local variable.
- The scope of these variables exists only within the block in which the variable is declared. i.e., we can access these variables only within that block.
- Initialization of the local variable is mandatory before using it in the defined scope.

Example:

```
class Add{
    public static void main(String[] args){
        int a=10, b=20; //Local variables
        System.out.println (a+b);
    }
}
```

```
    }  
}
```

2. Instance Variables

- Instance variables are non-static variables and are declared in a class outside any method, constructor, or block.
- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Use access specifier for instance variables. If we do not specify any access specifier, then the default access specifier will be used.
- Initialization of Instance Variable is not mandatory. Its default value is 0
- Instance Variable can be accessed only by creating objects.

Example:

```
class Add{  
    int a=10, b=20; // Instance variables  
    public static void main(String[] args){  
        // Instance or object creation  
        Add ob = new Add();  
  
        // Accessing Instance Variable using object  
        System.out.println (ob.a+ob.b);  
    }  
}
```

3. Static Variables

- Static variables are also known as Class variables.
- These variables are declared similarly as instance variables. The difference is that static variables are declared using the static keyword within a class outside any method constructor or block.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Initialization of Static Variable is not Mandatory. Its default value is 0
- we can access the static variable like the Instance variable (through an object).
- we can access the static variable with class name.
- we can access the static variable without class name.

Example:

```
class Add{
    static int a=10, b=20; // static variables
    public static void main(String[] args){

        // Instance or object creation
        Add ob = new Add();

        //Accessing static variables without class name
        System.out.println (a+b);

        //Accessing static variables with class name
        System.out.println(Add.a+Add.b);

        //Accessing static variables with object
        System.out.println(ob.a+ob.b);

    }
}
```

Diff b/w Instance Variables and Static Variables:**Instance Variables:**

- Instance Variables are declared without static keyword.
- They are specific to object i.e., for every object, separate memory will be allocated for each instance Variable.
- They can be accessed only by using the Object.

Static Variables:

- Static Variables are declared with static keyword.
- They are also called as “Class Variables”.
- They are not specific to any object.
- The memory for static Variables will be allocated whenever the class is loaded in to the memory i.e., memory will be allocated only once which will be shared by all the objects of the class.

Data types in Java:

- The type of value stored in a variable is called as datatype.

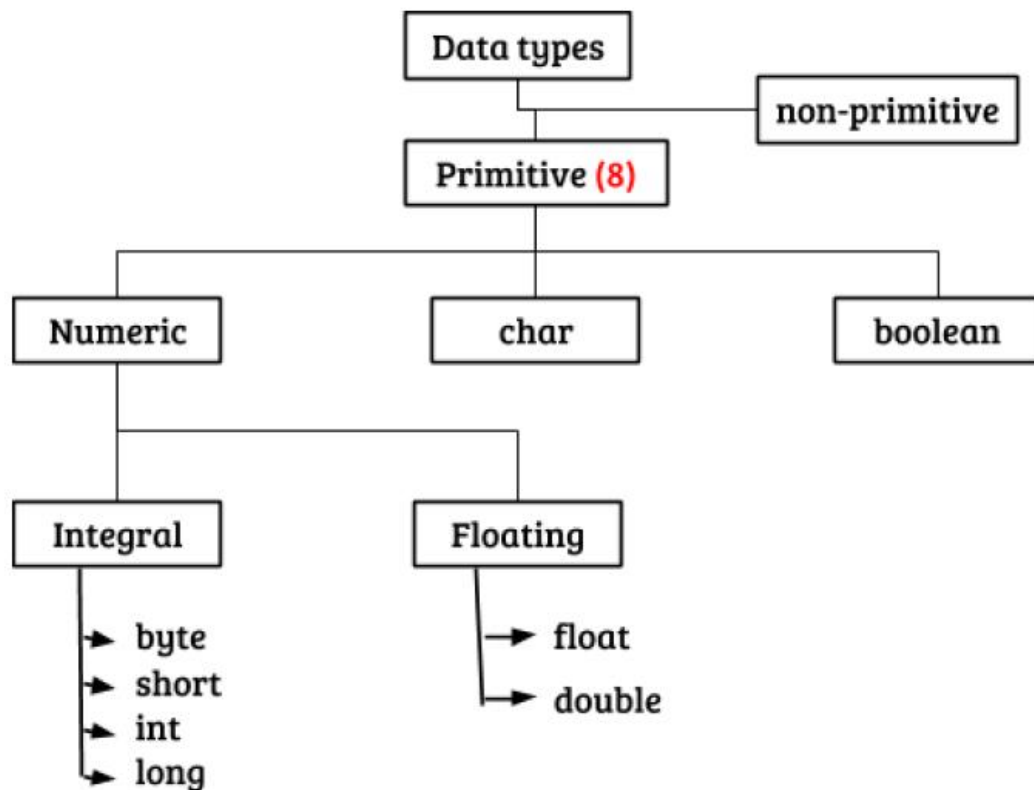
Ex: int a = 20;

char ch= 'A';

Data types can be classified into two types in java

1. Primitive Data Types

2. Non-Primitive Data Types



Note:

Except Boolean and char all remaining data types are considered as signed data types because we can represent both “+ve” and “-ve” numbers.

1. Primitive Data Types:

- There are 8 primitive data types supported by Java.
- Primitive data types are predefined by the language and named by a key word.

A. byte:

- ❖ Size: 1 Byte (8 bits)
- ❖ Max-value: +127
- ❖ Min-value: -128
- ❖ Range: -128 to 127 [-2^7 to $2^7 - 1$]

Example:

byte b=10;	▶ ✓
byte b2=130;	▶ C.E: possible loss of precision
byte b=10.5	▶ C.E: possible loss of precision
byte b=true;	▶ C.E: incompatible types
byte b="REC";	▶ C.E: incompatible types

- byte data type is best suitable if we are handling data in terms of streams either from the file or from the network.

B. short:

- ❖ Size: 2 Bytes
- ❖ Range: -32768 to 32767 (-2^{15} to $2^{15}-1$)

Example:

short s=130; ► ✓
short s=32768; ► C.E: possible loss of precision
short s=true; ► C.E: incompatible types

- The most rarely used data type in java is short.
- Short data type is best suitable for 16-bit processors like 8086 but these processors are completely outdated and hence the corresponding short data type is also outdated data type.

C. Int:

- ❖ Size: 4 bytes
- ❖ Range: -2147483648 to 2147483647 (-2^{31} to $2^{31}-1$)

Example:

int i=130; ► ✓
int i=10.5; ► C.E: possible loss of precision
int i=true; ► C.E: incompatible types

- This is most commonly used data type in java.

D. long:

- ❖ Size: 8 bytes
- ❖ Range: -2^{63} to $2^{63}-1$

Example:

long l= 13l;

➤ To hold the no. Of characters present in a big file int may not enough hence the return type of length() method is long.
long l=f.length() ; ► f is a file

- Whenever int is not enough to hold big values then we should go for long data type.

Note:

All the above data types (byte, short, int and long) can be used to represent whole numbers. If we want to represent real numbers then we should go for floating point data types.

E. Floating Point Data types:

float	double
If we want to 5 to 6 decimal places of accuracy then we should go for float.	If we want to 14 to 15 decimal places of accuracy then we should go for double.
Size: 4 bytes	Size: 8 bytes
Range: -3.4e38 to 3.4e38	Range: -1.7e308 to 1.7e308.

F. boolean data type:

- ❖ Size: Not applicable (virtual machine dependent)
- ❖ Range: Not applicable but allowed values are true or false.

Example1:

```
boolean b=true;      ► ✓
boolean b=True;      ► C.E: cannot find symbol
boolean b="True";    ► C.E: incompatible types
boolean b=0;         ► C.E: incompatible types
```

Example 2:

```
int x=10;
if( x )
    System.out.println("Hello");
else
    System.out.println("HI");
    ► Compiler error: incompatible types
    Found: int
    Require: boolean
```

G. char data type:

- ❖ Size: 2 bytes
- ❖ Range: 0 to 65535

Example:

```
char ch1=97;      ► ✓
char ch2=65536;   ► C.E: possible loss of precision
```

➤ In java we are allowed to use any worldwide alphabets character and java is Unicode based to represent all these characters one byte is not enough compulsory we should go

for 2 bytes.

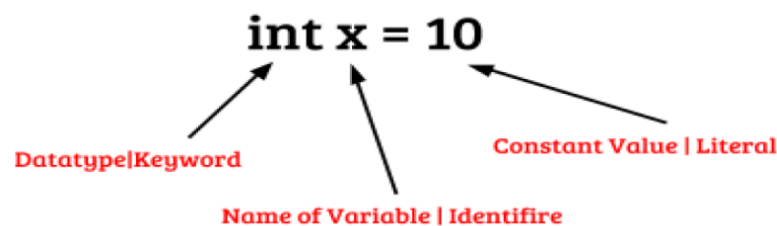
Summary of java primitive data types:

Data Type	Default Value	Size	Range
byte	0	1 byte	-2^7 to 2^7-1 (-128 to 127)
short	0	2 byte	-2^{15} to $2^{15}-1$ (-32768 to 32767)
int	0	4 byte	-2^{31} to $2^{31}-1$ (-2147483648 to 2147483647)
long	0L	8 byte	-2^{63} to $2^{63}-1$
float	0.0f	4 byte	$-3.4e^{38}$ to $3.4e^{38}$
double	0.0d	8 byte	$-1.7e^{308}$ to $1.7e^{308}$
char	Blank Space	2 byte	0 to 65535
boolean	false	1 bit	true / false

Literals in Java:

Literal: Any constant value which can be assigned to the variable is called literal/constant.

Example:



Literals in Java can be classified into six types, as below:

1. Integral Literals
2. Floating-point Literals
3. Char Literals
4. String Literals
5. Boolean Literals
6. Null Literals

1. Integral Literals

Integral literals are specified in four different ways, as follows:

Decimal: It has base ten, and digits from 0 to 9.

For example: int x = 108;

Octal:

It has base eight and allows digits from 0 to 7. While assigning an octal literal in the Java code, a number must have a prefix 0.

For example: `int x = 0745;`

Hexadecimal:

It has base 16. Hexadecimal allows digits from 0 to 9, and characters from A to F. Even though Java is case sensitive, and it also provides an exception for using either uppercase or lowercase characters in the code for hexadecimal literals.

For example:

```
int x = 0X123Fadd;  
int x = 0xFACE;  
int x = 0xFace;
```

Binary:

It can be specified in binary literals, that is 0 and 1 with a prefix 0b or 0B.

For example: `int x = 0b1011;`

Note:

There is no direct way to specify **byte** and **short** literals explicitly. But whenever we are assigning integral literal to the byte variables and its value within the range of byte compiler automatically treats as byte literal. Similarly short literal also.

- | | | |
|-------------------|-----------|-----------------------------------|
| 1. byte b=10; | ▶ valid | |
| 2. byte b=130; | ▶ Invalid | ▶ C.E: possible loss of precision |
| 3. short s=32767; | ▶ valid | |
| 4. short s=32768; | ▶ Invalid | ▶ C.E: possible loss of precision |

2. Floating Point Literals:

▶ Floating point literal is by default double type but we can specify explicitly as float type by suffixing with f or F.

Example:

- | | | |
|----------------------|-----------|-----------------------------------|
| 1. float f=123.456; | ▶ Invalid | ▶ C.E: possible loss of precision |
| 2. float f=123.456f; | ▶ valid | |
| 3. double d=123.456; | ▶ valid | |

▶ We can specify explicitly floating-point literal as double type by suffixing with d or D.

Example:

- | |
|-----------------------|
| 1. double d=123.456D; |
|-----------------------|

▶ We can specify floating point literal even in exponential form also(significant notation).

Example:

```
double d=10e2;      ► valid
System.out.println(d); ► Output: 1000.0
float f=10e2;       ► Invalid          ► C.E: possible loss of precision
float f=10e2F;      ► valid
```

3.Char literals:

- A char literal can be represented as single character within single quotes.

Example:

```
1. char ch='a';      ► valid
2. char ch=a;        ► Invalid          ► C.E: cannot find symbol a
3. char ch="a";      ► Invalid          ► C.E: incompatible types
4. char ch='ab';     ► Invalid          ► C.E: unclosed character literal
```

- We can specify a char literal as integral literal which represents Unicode of that character. We can specify that integral literal either in decimal or octal or hexadecimal form but allowed values range is 0 to 65535.

Example:

```
1. char ch=97;       ► valid
2. char ch=0xFace;   ► valid
3. char ch=65536;    ► Invalid          ► C.E: possible loss of precision
```

- We can represent a char literal by Unicode representation which is nothing but ‘\uxxxx’.

Example:

```
1. char ch1='\u0061';
System.out.println(ch1);
► Output: a
2. char ch2='\u0062; ► Invalid          ► C.E: cannot find symbol
3. char ch3='\iface'; ► Invalid          ► C.E: illegal escape character
```

- Every escape character in java acts as a char literal.

Example:

```
1. char ch='\n';     ► valid
2. char ch='\l';     ► Invalid          ► C.E: illegal escape character
```

4. String literals:

- Any sequence of characters with in double quotes is treated as String literal.

Example:

```
String s="Raghu";    ► Valid
```

5. Boolean literals:

- The only allowed values for the boolean type are true (or) false where case is important.

Example:

- | | | |
|--------------------|-----------|--------------------------------|
| 1. boolean b=true; | ➤ valid | |
| 2. boolean b=0; | ➤ Invalid | ➤ C.E: incompatible types |
| 3. boolean b=True; | ➤ Invalid | ➤ C.E: cannot find symbol True |
| 4. b="true"; | ➤ Invalid | ➤ C.E: incompatible types |

6. Null Literals

- Null literal in Java representing a null value.
- Null often describe the uninitialized state in the program.
- Th is value refers to no object.

Operators in Java:

Operator in Java is a symbol that is used to perform operations on variables / operands / constants.

Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups:

1. Arithmetic Operators
2. Relational Operators
3. Bitwise Operators
4. Logical Operators
5. Assignment Operators
6. Unary Operators
7. Conditional Operator
8. Misc Operators

1. Arithmetic Operators

Java arithmetic operators are used to perform addition, subtraction, multiplication, and division. They act as basic mathematical operations.

The following table lists the arithmetic operators and Assume integer variable A holds 10 and variable B holds 20, then

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	A + B will give 30
- (Subtraction)	Subtracts right-hand operand from left-hand operand.	A - B will give -10
* (Multiplication)	Multiplies values on either side of the operator.	A * B will give 200
/ (Division)	Divides left-hand operand by right-hand operand.	B / A will give 2
% (Modulus)	Divides left-hand operand by right-hand operand and returns remainder.	B % A will give 0

Example

```

public class Test {
    public static void main(String args[]) {
        int a = 10;
        int b = 20;
        int c = 25;
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("b / a = " + (b / a));
        System.out.println("b % a = " + (b % a));
        System.out.println("c % a = " + (c % a));
    }
}

```

Output

```

a + b = 30
a - b = -10
a * b = 200
b / a = 2
b % a = 0
c % a = 5

```

2. Relational Operators

Relational operators are used to check the relationship between two operands.

The following table lists the Relational operators and Assume integer variable A holds 10 and variable B holds 20, then

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Example

```
public class Test {

    public static void main(String args[]) {
        int a = 10;
        int b = 20;

        System.out.println("a == b = " + (a == b));
        System.out.println("a != b = " + (a != b));
        System.out.println("a > b = " + (a > b));
        System.out.println("a < b = " + (a < b));
        System.out.println("b >= a = " + (b >= a));
        System.out.println("b <= a = " + (b <= a));
    }
}
```

Output

```
a == b = false
a != b = true
a > b = false
a < b = true
b >= a = true
b <= a = false
```

3. Bitwise Operators

Bitwise operators in Java are used to perform operations on individual bits.

The following table lists the bitwise operators and Assume integer variable A holds 60 and variable B holds 13 then

Operator	Description	Example
& (bitwise and)	The bitwise AND & operator returns 1 if and only if both the operands are 1. Otherwise, it returns 0.	(A & B) will give 12 which is 0000 1100
(bitwise or)	The bitwise OR operator returns 1 if at least one of the operands is 1. Otherwise, it returns 0.	(A B) will give 61 which is 0011 1101
^ (bitwise XOR)	The bitwise XOR ^ operator returns 1 if and only if one of the operands is 1. However, if both the operands are 0 or if both are 1, then the result is 0.	(A ^ B) will give 49 which is 0011 0001
~ (bitwise compliment)	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<< (left shift)	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>> (right shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 1111
>>> (Unsigned Right Shift)	Unsigned Right Shift operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>>2 will give 15 which is 0000 1111

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int a = 60; /* 60 = 0011 1100 */  
        int b = 13; /* 13 = 0000 1101 */  
        int c = 0;  
  
        c = a & b;    /* 12 = 0000 1100 */  
        System.out.println("a & b = " + c);  
  
        c = a | b;    /* 61 = 0011 1101 */  
        System.out.println("a | b = " + c);  
  
        c = a ^ b;    /* 49 = 0011 0001 */  
        System.out.println("a ^ b = " + c);  
  
        c = ~a;       /* -61 = 1100 0011 */  
        System.out.println("~a = " + c);  
  
        c = a << 2;    /* 240 = 1111 0000 */  
        System.out.println("a << 2 = " + c);  
  
        c = a >> 2;    /* 15 = 1111 */  
        System.out.println("a >> 2 = " + c);  
  
        c = a >>> 2;   /* 15 = 0000 1111 */  
        System.out.println("a >>> 2 = " + c);  
    }  
}
```

Output

```
a & b = 12  
a | b = 61  
a ^ b = 49  
~a = -61  
a << 2 = 240  
a >> 2 = 15  
a >>> 2 = 15
```

4. Logical Operators:

Logical operators are used to check whether an expression is true or false. They are used in decision making.

The following table lists the logical operators and Assume Boolean variables A holds true and variable B holds false, then

Operator	Description	Example
&& (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
(logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A B) is true
! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true

Example:

```
public class Test {
    public static void main(String args[]) {
        boolean a = true;
        boolean b = false;
        System.out.println("a && b = " + (a&&b));
        System.out.println("a || b = " + (a||b));
        System.out.println("! (a && b) = " + ! (a && b));
    }
}
```

Output

```
a && b = false
a || b = true
!(a && b) = true
```

5. Assignment Operators:

Assignment operators are used in Java to assign values to variables.

Following are the assignment operators supported by Java language.

Operator	Example	Equivalent to
=	a = b;	a = b;
+=	a += b;	a = a + b;
-=	a -= b;	a = a - b;
*=	a *= b;	a = a * b;
/=	a /= b;	a = a / b;
%=	a %= b;	a = a % b;

Example:

```
public class Test {

    public static void main(String args[]) {
        int a = 10;
        int b = 20;
        int c = 0;

        c = a + b;
        System.out.println("c = a + b = " + c);

        c += a;
        System.out.println("c += a = " + c);
        c -= a;
        System.out.println("c -= a = " + c);

        c *= a;
        System.out.println("c *= a = " + c);

        a = 10;
        c = 15;
        c /= a;
        System.out.println("c /= a = " + c);

        a = 10;
        c = 15;
        c %= a;
        System.out.println("c %= a = " + c);
    }
}
```

Output

```
c = a + b = 30
c += a = 40
c -= a = 30
c *= a = 300
c /= a = 1
c %= a = 5
```

6. Unary Operators

In Java, the **unary operator** is an operator that can be used only with an operand. It is used to represent the **positive** or **negative** value, **increment/decrement** the value by 1, and **complement** a Boolean value.

The following table describes the short description of the unary operators.

Operator Name	Symbol	Description	Example	Equivalent Expression
Unary Plus	+	It is used to represent the positive value.	+a	a
Unary Minus	-	It is used to represent the negative value.	-a	-
Increment Operator	++	It increments the value of a variable by 1 .	++a or a++	a=a+1
Decrement Operator	--	It decrements the value of a variable by 1 .	--a or a--	a=a-1
Logical Complement Operator	!	It inverts the value of a boolean variable.	!true	-

Example 1: Java Unary Operator Example: ++ and --

```
public class OperatorExample{
    public static void main(String args[]){
        int x=10;
        System.out.println(x++);
        System.out.println(++x);
        System.out.println(x--);
        System.out.println(--x);
    }
}
```

Output

```
10
12
12
10
```

Example 2: Java Unary Operator Example: ~ and !

public class OperatorExample{	
public static void main(String args[]){	Output
int a=10;	-11
int b=-10;	9
boolean c=true;	false
boolean d=false;	true
System.out.println(~a);	
System.out.println(~b);	
System.out.println(!c);	
System.out.println(!d);	
}	
}	

7. Conditional Operator:

The conditional operator is also called a **ternary operator** because it requires three operands. This operator is used for decision making. In this operator, first, we verify a condition, then we perform one operation out of the two operations based on the condition result. If the condition is TRUE the first option is performed, if the condition is FALSE the second option is performed.

Syntax:

Condition ? TRUE Part : FALSE Part;

Example:

```
public class ConditionalOperator {

    public static void main(String[] args) {

        int a = 10, b = 20, c;

        c = (a>b)? a : b;

        System.out.println("c = " + c);

    }

}
```

Output:

c = 20

8. Misc Operators

Instanceof operator:

The **java instanceof operator** is used to test whether the object is an instanceof the specified type (class or subclass or interface).

The instanceof in java is also known as type *comparison operator* because it compares the instance with type. It returns either true or false. If we apply the instanceof operator with any variable that has null value, it returns false.

Example:

```
class Simple1{
    public static void main(String args[]){
        Simple1 s=new Simple1();
        System.out.println(s instanceof Simple1);
    }
}
```

Output:

true

Expressions in Java:

- An expression is a collection of **operators** and **operands** that represents a specific value.
- In the above definition, **an operator** is a symbol that performs tasks like arithmetic operations, logical operations, and conditional operations, etc.
- **Operands** are the values on which the operators perform the task. Here operand can be a direct value or variable or address of memory location.

Following are some of the examples for expressions in Java:

```
int a = 10; //Assignment expression

System.out.println("Value = "+x");

int result = a + 10; //Assignment exp

if(val1 <= val2) //Boolean expression

b = a++; //Assignment exp
```

Expression evaluation in Java is based upon the following concepts:

- Operator precedence
- Associativity rules

Precedence rules and Associativity**Operator precedence:**

All the operators in Java are divided into several groups and are assigned a precedence level. The operator precedence and associativity rules for the operators in Java is shown below:

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

Operator precedence determines which operator is performed first in an expression with more than one operator with different precedence.

Example:

```
public class Precedence {
    public static void main(String[] args) {
        System.out.println(3 + 5 * 5);
        System.out.println((3 + 5) * 5);
    }
}
```

Output:

```
28
40
```

Explanation for $3 + 5 * 5$ (Expression).

Based on the operator precedence table shown above, $*$ has higher precedence than $+$. So, $5 * 5$ is evaluated first which gives 25 and then $3 + 25$ is evaluated which gives 28.

Explanation for $(3 + 5) * 5$ (Expression).

Based on the operator precedence table shown above, $()$ has higher precedence than $*$. So, $3 + 5$ is evaluated first which gives 8 and then $8 * 5$ is evaluated which gives 40.

Associativity:

Operators Associativity is used when two operators of same precedence appear in an expression. Associativity can be either Left to Right or Right to Left.

For example: $*$ and $/$ have same precedence and their associativity is Left to Right

public class Precedence {

```
    public static void main(String[] args) {  
        System.out.println(10 / 5 * 5);  
    }  
}
```

Output:

10

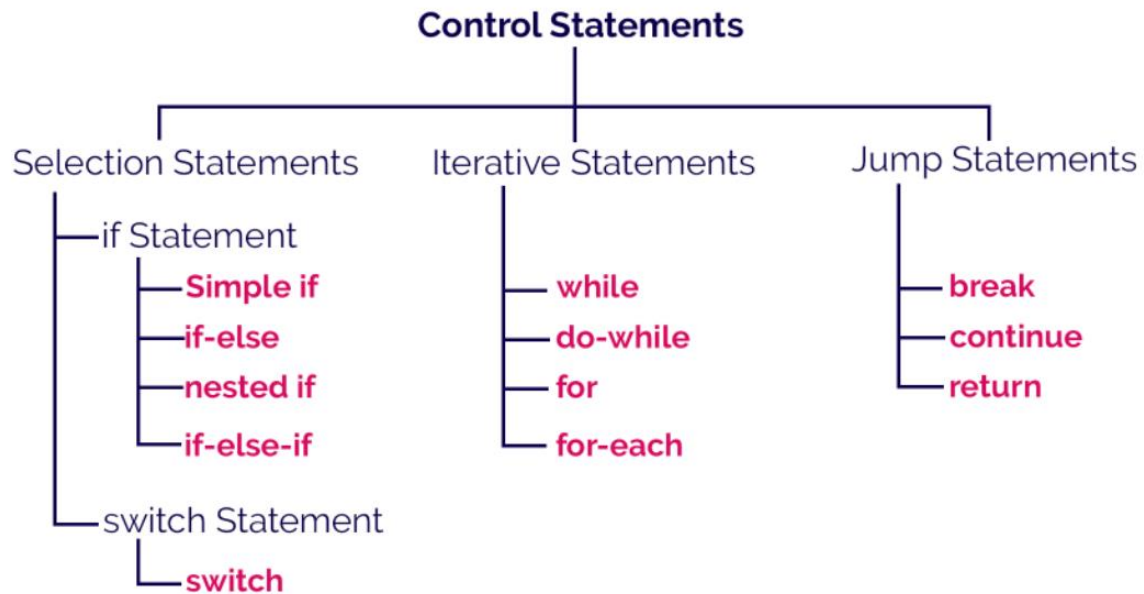
In the above expression, the operators $/$ and $*$ both belong to the same group in the operator precedence table. So, we have to check the associativity rules for evaluating the above expression. Associativity rule for $/$ and $*$ group is left-to-right i.e, evaluate the expression from left to right. So, $10/5$ is evaluated to 2 and then $2*5$ is evaluated to 10.

Primitive type conversion and casting:**Difference Between Type Casting and Type Conversion**

S.N.	Type Conversion	Type Casting
1	Type conversion allows a compiler to convert one data type to another data type at the compile time of a program or code.	Type casting is a mechanism in which one data type is converted to another data type using a casting () operator by a programmer.
2	Type conversion is only used with compatible data types, and hence it does not require any casting operator.	It can be used both compatible data type and incompatible data type.
3	It does not require any programmer intervention to convert one data type to another type.	It requires a programmer to manually casting one data into another type.
4	It is used or take place at the compile time of a program.	It is used while designing a program by the programmer.
5	When converting one data type to another, the destination type should be greater than the source data type.	When casting one data type to another, the destination data type must be smaller than the source data.
6	It is also known as widening conversion or down casting because one smaller data type converts to a larger data type.	It is also known as narrowing casting or up casting because one larger data type converts to a smaller data type.
7	It is also known as implicit type conversion.	It is also known as explicit type conversion.
8	In type conversion, data is unlikely to be lost when converting from a small to a large data type.	There is a possibility of data or information being lost in type casting.
9	<pre>int a=10; double d=a; System.out.println(d);</pre>	<pre>double d=10.20; int i=(int)d; System.out.println(i);</pre>

Flow of Control in Java:

In java, the default execution flow of a program is a sequential order. But the sequential order of execution flow may not be suitable for all situations. Sometimes, we may want to jump from line to another line, we may want to skip a part of the program, or sometimes we may want to execute a part of the program again and again. To solve this problem, java provides control flow statements.



The control statements are used to control the order of execution according to our requirements. Java provides several control statements, and they are classified as follows.

Types of Control Statements

In java, the control statements are classified as follows.

- ❖ Selection Control Statements (Decision Making Statements)
- ❖ Iterative Control Statements (Looping Statements)
- ❖ Jump Statements

Selection Control Statements

In java, the selection statements are also known as decision making statements or branching statements. The selection statements are used to select a part of the program to be executed based on a condition. Java provides the following selection statements.

- ❖ if statement
- ❖ if-else statement
- ❖ if-else-if ladder statement
- ❖ nested if statement
- ❖ switch statement

❖ if statement

The if statement checks, the given condition then decides the execution of a block of statements. If the condition is True, then the block of statements is executed and if it is False, then the block of statements is ignored.

Syntax:

```
if(condition){  
    //code to be executed  
}
```

Example:

```
class IfStatement {  
    public static void main(String[] args) {  
        int number = 10;  
        if (number < 0) {  
            System.out.println("The number is negative.");  
        }  
  
        System.out.println("Statement outside if block");  
    }  
}
```

Output:

Statement outside if block

❖ if-else Statement

The Java if-else statement also tests the condition. It executes the *if block* if condition is true otherwise *else block* is executed.

Syntax:

```
if(condition){  
    //code if condition is true  
}else{  
    //code if condition is false  
}
```

Example:

```
class Main {  
    public static void main(String[] args) {  
        int number = 10;  
        if (number > 0)  
            System.out.println("The number is positive.");  
        else  
            System.out.println("The number is not positive.");  
    }  
}
```

Output:

The number is positive

❖ if-else-if ladder Statement

The if-else-if ladder statement executes one condition from multiple statements.

Syntax:

```
if(condition1){
//code to be executed if condition1 is true
}else if(condition2){
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

Example:

```
public class IfElseIfExample {
public static void main(String[] args) {
    int marks=65;

    if(marks<50){
        System.out.println("fail");
    }
    else if(marks>=50 && marks<60){
        System.out.println("D grade");
    }
    else if(marks>=60 && marks<70){
        System.out.println("C grade");
    }
    else if(marks>=70 && marks<80){
        System.out.println("B grade");
    }
    else if(marks>=80 && marks<90){
        System.out.println("A grade");
    }else if(marks>=90 && marks<100){
        System.out.println("A+ grade");
    }else{
        System.out.println("Invalid!");
    }
}
}
```

Output:

C grade

❖ nested if statement

The nested if statement represents the *if block within another if block*. Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
if(condition){  
    //code to be executed  
    if(condition){  
        //code to be executed  
    }  
}
```

Example:

```
public class JavaNestedIfExample {  
    public static void main(String[] args) {  
        //Creating two variables for age and weight  
        int age=20;  
        int weight=80;  
        //applying condition on age and weight  
        if(age>=18){  
            if(weight>50){  
                System.out.println("You are eligible to donate blood");  
            }  
        }  
    }  
}
```

Output:

You are eligible to donate blood

❖ Switch Statement

The *switch statement* executes one statement from multiple conditions. It is like if-else-if ladder statement.

Points to Remember

- ❖ There can be *one or N number of case values* for a switch expression.
- ❖ The case value must be of switch expression type only. The case value must be *literal or constant*. It doesn't allow variables.
- ❖ The case values must be *unique*. In case of duplicate value, it renders compile-time error.
- ❖ The Java switch expression must be of *byte, short, int, long (with its Wrapper type), enums and string*.
- ❖ Each case statement can have a *break statement* which is optional. When control reaches to the break statement, it jumps the control after the switch expression. If a break statement is not found, it executes the next case.
- ❖ The case value can have a *default label* which is optional.

Syntax:

```
switch(expression){  
case value1:  
    //code to be executed;  
    break; //optional  
case value2:  
    //code to be executed;  
    break; //optional  
.....  
default:  
    code to be executed if all cases are not matched;  
}
```

Example:

```
public class SwitchExample {  
public static void main(String[] args) {  
    //Declaring a variable for switch expression  
    int number=20;  
    //Switch expression  
    switch(number){  
    //Case statements  
    case 10: System.out.println("10");  
    break;  
    case 20: System.out.println("20");  
    break;  
    case 30: System.out.println("30");  
    break;  
}
```

```
//Default case statement
default: System.out.println("Not in 10, 20 or 30");
}
}
}
```

Output:

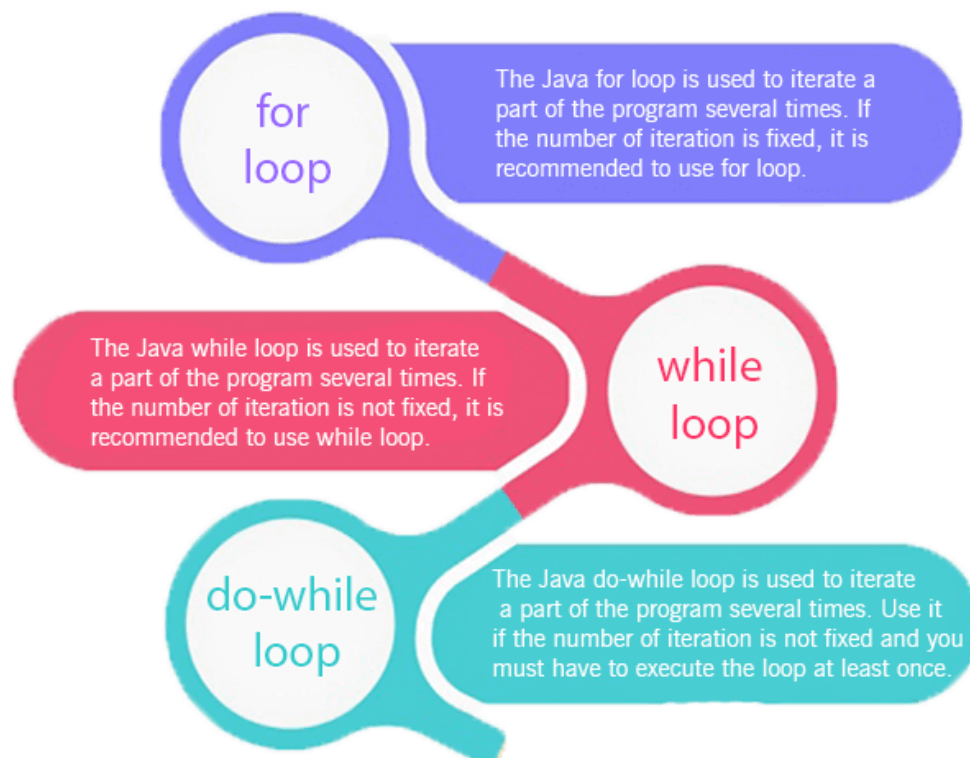
20

❖ Iterative Control Statements (Looping Statements)

In Java *loop* is used to iterate a part of the program several times.

A simple for loop is the same as C/C++. We can initialize the variable, check condition and increment/decrement value. It consists of four parts:

1. **Initialization:** It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.
2. **Condition:** It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return boolean value either true or false. It is an optional condition.
3. **Increment/Decrement:** It increments or decrements the variable value. It is an optional condition.
4. **Statement:** The statement of the loop is executed each time until the second condition is false.



for-each loop

- ❖ The Java for-each statement was introduced since Java 5.0 version.
- ❖ It provides an approach to traverse through an array or collection in Java.
- ❖ The for-each statement also known as enhanced for statement.
- ❖ The for-each statement executes the block of statements for each element of the given array or collection.

Syntax:**For**

```
for(initialization; condition; increment/decrement){  
    //statement or code to be executed  
}
```

While

```
initialization  
while (condition){  
    //code to be executed  
    Increment / decrement statement  
}
```

Do-While

```
initialization  
do{  
    //code to be executed / loop body  
    //update statement  
}while (condition);
```

For-each Loop

```
for(data_type variable : array_name){  
    //code to be executed  
}
```

Examples:**ForExample.java**

//Java Program to demonstrate the example of for loop

```
public class ForExample {  
    public static void main(String[] args) {  
        //Code of Java for loop  
        for(int i=1;i<=10;i++){  
            System.out.println(i);  
        }  
    }  
}
```

//Java Program to demonstrate the example of While loop

WhileExample.java

```
public class WhileExample {  
    public static void main(String[] args) {  
        int i=1;  
        while(i<=10){  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```

//Java Program to demonstrate the example of Do-While loop

DoWhileExample.java

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i=1;  
        do{  
            System.out.println(i);  
            i++;  
        }while(i<=10);  
    }  
}
```

Output:

1
2
3
4
5
6
7
8
9
10

Infinite Loop:**ForExample.java**

```
//Java program to demonstrate the use of infinite for loop
```

```
public class ForExample {
    public static void main(String[] args) {
        //Using no condition in for loop
        for(;;){
            System.out.println("infinitive loop");
        }
    }
}
```

```
//Java program to demonstrate the use of infinite While loop
```

WhileExample.java

```
public class WhileExample2 {
    public static void main(String[] args) {
        // setting the infinite while loop by passing true to the condition
        while(true){
            System.out.println("infinitive loop");
        }
    }
}
```

```
//Java program to demonstrate the use of infinite Do-While loop
```

DoWhileExample2.java

```
public class DoWhileExample2 {
    public static void main(String[] args) {
        do{
            System.out.println("infinitive loop");
        }while(true);
    }
}
```

o/p

Infinite Loop

Nested for Loop

If we have a for loop inside another loop, it is known as nested for loop. The inner loop executes completely whenever outer loop executes.

Example: PyramidExample.java

```
public class PyramidExample {  
    public static void main(String[] args) {  
        for(int i=1;i<=5;i++){  
            for(int j=1;j<=i;j++){  
                System.out.print("* ");  
            }  
            System.out.println();//new line  
        }  
    }  
}
```

Output:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Example: ForEachExample.java

```
//Java For-each loop example which prints the  
//elements of the array  
public class ForEachExample {  
    public static void main(String[] args) {  
        //Declaring an array  
        int arr[]={12,23,44,56,78};  
        //Printing array using for-each loop  
        for(int i:arr){  
            System.out.println(i);  
        }  
    }  
}
```

Output:

```
12  
23  
44  
56  
78
```

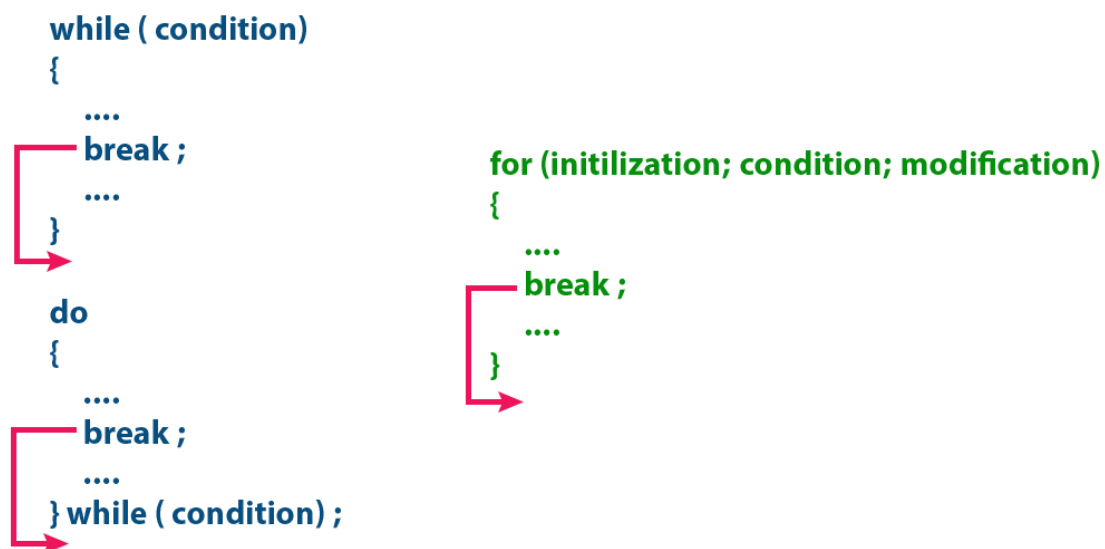
Jump Statements

The java programming language supports jump statements that used to transfer execution control from one line to another line. The java programming language provides the following jump statements.

- ❖ break statement
- ❖ continue statement
- ❖ labelled break and continue statements

break statement

The break statement in java is used to terminate a switch or looping statement. That means the break statement is used to come out of a switch statement and a looping statement like while, do-while, for, and for-each.



Example:

```
public class JavaBreakStatement {

    public static void main(String[] args) {

        int list[] = {10, 20, 30, 40, 50};

        for(int i : list) {
            if(i == 30)
                break;
            System.out.println(i);
        }

    }

}
```

Output:

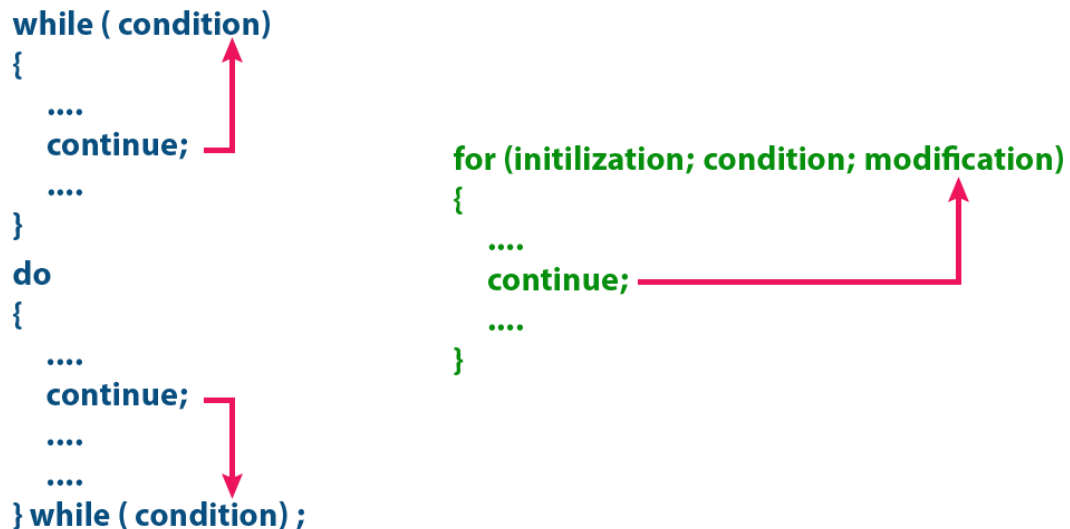
10

20

Continue Statement

The Java *continue statement* is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition. In case of an inner loop, it continues the inner loop only.

We can use Java continue statement in all types of loops such as for loop, while loop and do-while loop.



Example: ContinueExample.java

```
public class ContinueExample {
    public static void main(String[] args) {
        for(int i=1;i<=5;i++){
            if(i==3){
                continue;    //it will skip the rest statement
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
4
5
```

❖ labelled break and continue statements

- ❖ The java programming language does not support **goto** statement, alternatively, the break and continue statements can be used with label.
- ❖ The labelled break statement terminates the block with specified label.
- ❖ The labelled continue statement takes the execution control to the beginning of a loop with specified label.

Example: LabeledForExample.java

//A Java program to demonstrate the use of labelled for loop using break statement.

```
public class LabeledForExample {  
    public static void main(String[] args) {  
        //Using Label for outer and for loop  
        aa:  
        for(int i=1;i<=3;i++){  
            bb:  
            for(int j=1;j<=3;j++){  
                if(i==2&&j==2){  
                    break aa;  
                }  
                System.out.println(i+" "+j);  
            }  
        }  
    }  
}
```

Output:

```
1 1  
1 2  
1 3  
2 1
```

Example: LabeledForExample.java

//A Java program to demonstrate the use of labelled for loop using continue statement.

```
public class LabeledForExample {  
    public static void main(String[] args) {  
        //Using Label for outer and for loop  
        aa:  
        for(int i=1;i<=3;i++){  
            bb:  
            for(int j=1;j<=3;j++){  
                if(i==2&& j==2){  
                    continue aa;  
                }  
                System.out.println(i+" "+j);  
            }  
        }  
    }  
}
```

Output:

```
1 1  
1 2  
1 3  
2 1  
3 1  
3 2  
3 3
```